2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

A Skeleton-Based Topological Planner for Exploration in Complex Unknown Environments

Haochen Niu, Xingwu Ji, Lantao Zhang, Fei Wen, Rendong Ying and Peilin Liu*

Abstract— **The capability of autonomous exploration in complex, unknown environments is important in many robotic applications. While recent research on autonomous exploration have achieved much progress, there are still limitations, e.g., existing methods relying on greedy heuristics or optimal path planning are often hindered by repetitive paths and high computational demands. To address such limitations, we propose a novel exploration framework that utilizes the global topology information of observed environment to improve exploration efficiency while reducing computational overhead. Specifically, global information is utilized based on a skeletal topological graph representation of the environment geometry. We first propose an incremental skeleton extraction method based on wavefront propagation, based on which we then design an approach to generate a lightweight topological graph that can effectively capture the environment's structural characteristics. Building upon this, we introduce a finite state machine that leverages the topological structure to efficiently plan coverage paths, which can substantially mitigate the back-and-forth maneuvers (BFMs) problem. Experimental results demonstrate the superiority of our method in comparison with state-of-the-art methods. The source code will be made publicly available at: https://github.com/Haochen-Niu/STGPlanner.**

I. INTRODUCTION

Recently, autonomous exploration has attracted much research attention due to its importance in many robotic applications such as search and rescue, environmental monitoring, and planetary exploration [1]. The primary objective of autonomous exploration is to enable robots to efficiently navigate and map unknown environments without human assistance or prior knowledge, thereby facilitating and enhancing subsequent mission tasks.

A key challenge in autonomous exploration is making optimal decision in environments with incomplete information. Robots must continuously optimize their paths to maximize information acquisition while minimizing resource consumption, such as time, energy, and computational power. Early works, such as frontier-based [2]–[4] or sampling-based methods [5]–[7], typically rely on greedy heuristics, which direct robots toward areas with the highest immediate information gain. However, due to the lack of a global perspective, these methods may result in low-quality trajectories, particularly in complex environments [8]–[10]. For instance, as the frontier of a local region shrinks during exploration, the robot may greedily shift toward another region with higher utility. As the utility in the new region diminishes, the robot returns to the previous region, leading to inefficient back-and-forth maneuvers (BFMs).

To address this issue, recent methods have integrated global and local information following a coarse-to-fine hierarchical paradigm, which plan globally optimal paths for exploration via constructing formulations akin to the Traveling Salesman Problem (TSP) [11]–[13]. However, due to the unpredictability of unknown spaces, these methods, based on partially observable information, still retain a greedy nature. Consequently, the BFMs problem can only be partially mitigated. Additionally, as the observed information changes during exploration, the optimal path needs to be recurrently replanned, which incurs a significant computational burden due to solving the TSP formulation frequently.

To address the aforementioned limitations, this work aims to leverage the environment's topology to guide exploration, rather than relying solely on maximum information gain or "global optimality". To this end, we propose STGPlanner, a novel autonomous exploration framework based on the skeletal topological graph (STG) of the environment. Specifically, we develop an incremental skeleton extraction method that captures the environment's topology from its observed geometry, forming the basis for generating the STG. This process implicitly integrates key elements such as frontiers, viewpoints, information gain, and path planning. Using this representation, we introduce high-level intention that prioritizes the complete exploration of the current STG branch to substantially mitigate inefficient BFMs. Building on this concept, a STG-based exploration strategy encapsulated within a finite state machine (FSM) is designed, which enables efficient exploration at a low computational cost.

The main contributions are as follows.

- A novel autonomous exploration framework, which utilizes the global topology information of observed environment to effectively address the BFMs problem, based on a STG representation of the environment geometry.
- A STG generation method based on incremental skeleton extraction, which can efficiently construct a concise topology of the observed environment with essential exploration information.
- A STG-based exploration strategy encapsulated within a FSM, which fully leverages the information of STG to enable efficient exploration decision at a low computational cost.
- Extensive experimental evaluation showcases the effectiveness of our method in various environments.

All authors are with the Brain-Inspired Application Technology Center (BATC), School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (email: {haochen_niu, jixingwu, swagger, wenfei, rdying, liupeilin}@sjtu.edu.cn).

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11128159

II. RELATED WORK

In the past decade, autonomous exploration has been extensively studied and many methods have been proposed. Typically, autonomous exploration involves three key stages: identification of candidates target/actions, utility evaluation, planning and execution [14].

The work [2] pioneered the frontier-based method, which introduces the concept of frontiers as the boundaries between known and unknown areas and uses frontiers as candidate targets, e.g., select the nearest frontier for exploration. This seminal work has inspired numerous subsequent improvements. For example, the work [3] incorporates field of view consideration to minimize speed variation and maintain high-speed movement. The work [15] proposes the frontier information structure (FIS) for rapid frontier detection, while [4] revisits frontier information gain through an automatic-differentiable function with respect to the robot's path.

Another class of methods for candidate target selection is sampling-based. RH-NBVP [5] first applies the next best view in a receding horizon manner to autonomous exploration. It spans a rapidly-exploring random tree (RRT) to calculate the information gain of each node via ray-casting and chooses the path with the highest utility, executing only the first step. Expanding on this foundation, the works [7], [16] introduce biased sampling to steer RRT growth towards unexplored regions, thereby improving exploration efficiency. Furthermore, [6] employs a dual-layer planning architecture and uses a dense rapidly-random graph to identify collision-free paths with maximal information gain.

However, these methods' reliance on greedy decisions often results in low-quality trajectories due to a lack of global perspective [10], [17]. To counter this, recent approaches integrate global and local information, using TSP formulations to optimize global trajectories based on current information. For example, the work [18] merges frontier and sampling methods, counting visible frontiers from sampled viewpoints as the information gain, reducing the computational overhead of ray-casting. It then uses the 2-opt algorithm to solve the TSP and find the best viewpoint order. Similarly, [8] uses occlusion-free spheres to improve viewpoint selection, treating it as an asymmetric TSP. To reduce computational complexity, some works combine a coarse global planning with a fine local planning. For example, the works [11], [12], [19] initially segment the map into grids to determine a coarse path via TSP, covering unexplored areas. Then, they sample viewpoints locally, compute information gain, and reapply TSP to refine the trajectory. Moreover, the work [13] replaces uniform grid with viewpoints clustering to achieve a more flexible global representation. Despite the efficiency improvement by incorporating global optimization, BFMs remain unavoidable due to the heuristic utility definitions and the partially observable information during exploration. Moreover, methods like [20] utilize semantic markers, such as door detection, to prioritize exploration of small regions, which introduce additional computational overhead and are restricted to specific environments.

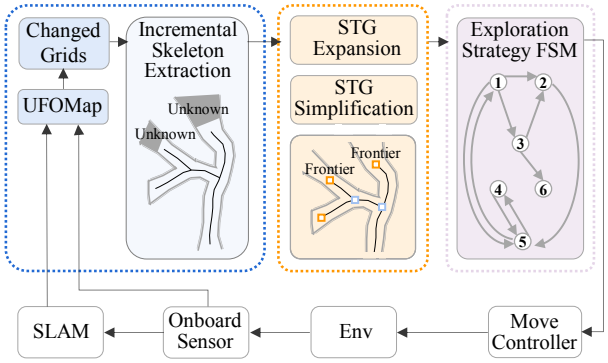


Fig. 1: An overview of the proposed STGPlanner.

To address these limitations, we utilize the environment's topology information to guide the robot in prioritizing comprehensive exploration within the current region, thereby reducing BFMs. Central to this concept, we introduce an incremental skeleton extraction method that generates a lightweight STG representing the observed environment with exploration information and design an FSM to implement STG-based exploration strategy, enabling highly efficient exploration with reduced computational overhead.

III. METHOD

A. System Overview

An overview of the proposed system framework is illustrated in Fig. 1, which consists of three primary modules: incremental skeleton extraction, STG update and an exploration strategy FSM. The Lidar SLAM module provides localization and registered point clouds, which are used by the 3D occupancy grid map UFOMap [21] to efficiently manage and record environment changes. Skeleton extraction (Sec. III-B) is performed incrementally using changed grids, allowing for fast updates. Based on the current exploration branch and updated skeleton, the STG expands and simplifies (Sec. III-C). The FSM then executes the exploration strategy by transitioning states according to the information of STG (Sec. III-D). This cycle repeats until the FSM reaches its termination state.

B. Incremental Skeleton Extraction

To reduce BFMs in exploration, we aim to leverage the topological structure of the observed environment to guide the robot's exploration behavior, which necessitates rapid extraction of a concise topological representation. The generalized Voronoi diagram (GVD) [22], which maximizes the distance between obstacles, provides a compact representation of the environment's geometry. Given that obstacles are represented by n convex point sets $C_i \subset \mathbb{R}^2$ (where $i = 1, 2, \ldots, n$), the two-equidistant face $\mathcal{F}_{ij}$ between sets C_i and C_j is defined as follows:

$$\begin{aligned} \mathcal{S}_{ij} &= \left\{ x \in \mathbb{R}^2 \mid d_i(x) = d_j(x),\ \nabla d_i(x) \neq \nabla d_j(x) \right\}, \\ \mathcal{F}_{ij} &= \left\{ x \in \mathcal{S}_{ij} \mid \forall k \neq i, j : d_i(x) \neq d_k(x) \right\}, \end{aligned} \tag{1}$$

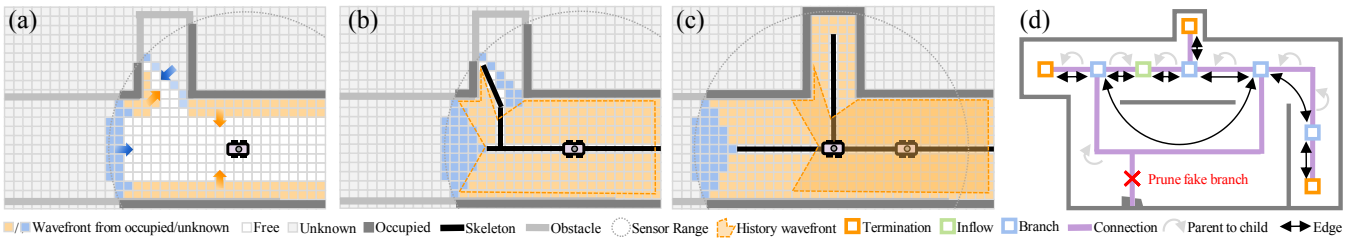


Fig. 2: Illustration of the skeleton extraction and STG update. (a) and (b) depict the wavefront propagation from unknown and occupied area at time T_i. As robot moves from T_i to T_{i+1} in (c), wavefront from occupied is preserved (shaded), thereby requiring only partial data processing. (d) provides an overview of the STG.

Algorithm 1: The Incremental Skeleton Extraction

Input: $Skeleton$, $waveFront$
Output: The updated skeleton $Skeleton$.

1 $Skeleton' \leftarrow Skeleton$;
2 $changed \leftarrow$ true;
3 $iteration \leftarrow 0$;
4 $phase \leftarrow 1$;
5 **while** $changed = true$ **do**
6 $changed \leftarrow$ false;
7 **for** $i = 0$ **to** 1 **do**
8 **for** *all* $s \in Skeleton$ **do**
9 **if** $waveFront_s > iteration$ ***or*** $state(Skeleton, s) \neq free$ **then**
10 **continue**;
11 **if** $(phase = 1$ ***and*** $(a) \wedge (b) \wedge (c) \wedge (d))$ ***or***
12 $(phase = 2$ ***and*** $(a) \wedge (b) \wedge (e) \wedge (f))$ **then**
13 $waveFront_s \leftarrow iteration$;
14 $changed \leftarrow$ true;
15 **if** $\exists n_i \in Adj_8(s)$ *such that* $state(Skeleton, n_i) = unknown$ **then**
16 $state(Skeleton', s) \leftarrow$ unknown;
17 **else**
18 $state(Skeleton', s) \leftarrow$ occupied;
19 $Skeleton \leftarrow Skeleton'$;
20 $iteration \leftarrow iteration + 1$;
21 $phase \leftarrow 3 - phase$;
22 `resetUnknownWaveFront();`
23 **return** $Skeleton$;

where $d_i(x)$ is the minimum distance from point x to set C_i, and $\nabla d_i(x)$ is the direction from x to its nearest point in C_i. The GVD is the union of all $\mathcal{F}_{ij}$, expressed as:

$$\mathcal{F}^2 = \cup_{i=1}^{n-1} \cup_{j=i+1}^{n} \mathcal{F}_{ij}. \tag{2}$$

However, extracting a strict GVD as per the definition incurs high computational complexity, and such precision is unnecessary for exploration tasks. Inspired by [23], we propose an approximate method to iteratively identify skeleton candidates and support incremental updates. Pseudo-code for incremental skeleton extraction is provided in Alg. 1. The 8-neighborhoods of $s(i, j)$, $Adj_8(s)$, consist of the points surrounding $s(i, j)$, with n_1 as the top neighbor and n_2 to n_8 indexed clockwise.

We iteratively process all changed free grids in the UFOMap until no grid satisfies the specific conditions. This process resembles wavefront propagation, where waves originate from free grids adjacent to occupied or unknown regions. The skeleton is formed at the points where two waves collide, as illustrated in Fig. 2. Following [23], each iteration is divided into two phases with distinct conditions (lines 11-12, Alg. 1), as described below:

$$\begin{array}{llll} (a) & A(s) = 1, & (b) & 2 \leq B(s) \leq 6, \\ (c) & n_1 * n_3 * n_5 = 0, & (d) & n_3 * n_5 * n_7 = 0, \\ (e) & n_1 * n_3 * n_7 = 0, & (f) & n_1 * n_5 * n_7 = 0, \end{array} \tag{3}$$

where $A(s)$ represents the number of transitions from 0 to 1 in the ordered sequence $n_1, n_2, \cdots, n_8$, and $B(s)$ denotes the number of nonzero neighbors of s, defined as:

$$B(s) = n_1 + n_2 + n_3 + \cdots + n_7 + n_8, \tag{4}$$

where $n_i = 1$ if $state(s_{ni}) = free$ otherwise $n_i = 0$.

To support incremental updates, the wavefront of each removed grid is recorded, and its state is marked based on the states of its neighboring grids (lines 13-18, Alg. 1). During each iteration, grids with wavefront values exceeding the current iteration count are skipped (line 9, Alg. 1). Upon completion, $waveFront$ values of unknown grids are reset to zero (line 22, Alg. 1). This approach eliminates redundant wavefront propagation from obstacles while addressing unknown region uncertainty, ensuring computational efficiency and skeleton continuity, as shown in Fig. 2. Additionally, the wavefront value provides insight into the path's width and is subsequently applied in Sec. III-C.2 and Sec. III-D.

C. Skeletal Topological Graph Update

The update process of the STG, denoted as $\mathcal{G} = (\mathcal{N}, \mathcal{E})$, is detailed in Alg. 2. To accurately document the exploration process and facilitate decision-making, the graph is constructed in a tree-growth-inspired manner, with nodes classified into four distinct types:

- **Termination** (N_T): One parent, no children.
- **Connection** (N_C): One parent, one child.
- **Branch Junction** (N_B): One parent, multiple children.
- **Inflow Junction** (N_I): Multiple parents, any children.

1) Expansion: During initialization, the skeleton grid closest to the starting point is marked as s_{Home}. **genNode**(s_{Home}) is called to generate the corresponding node $n_{Home} \in \mathcal{N}$, which is added to the frontier candidates

Algorithm 2: The Graph Generation

Input: $Skeleton$, $waveFront$, $\mathcal{G}$, $\mathcal{N}_f$
Output: The updated graph $\mathcal{G}$ and frontier set $\mathcal{N}_f$.

1 **Function** `expandRec(`n_s`)`**:**
2 $childQueue \leftarrow$ `empty;`
3 **for** *all* s_{ni} *in* $Adj_8(s)$ **do**
4 **if** $state(skeleton, s_{ni}) = unknown$ **then**
5 $\mathcal{N}_f \leftarrow \mathcal{N}_f \cup n_s$;
6 **else if** $state(skeleton, s_{ni}) = free$ **then**
7 **if** $n_{s_{ni}} \in \mathcal{N}$ **then**
8 $\mathcal{N}_i^2 \leftarrow \mathcal{N}_i^2 \cup \{(n_{s_{ni}}, n_s)\}$;
9 **else**
10 `genNode(`s_{ni}`);`
11 $childQueue \leftarrow childQueue \cup n_{s_{ni}}$;
12 `setNodeType(`n_s`);`
13 **for** *all* n'_s *in* $childQueue$ **do**
14 `expandRec(`n'_s`);`

`/* start main function */`
15 $\mathcal{N}'_f \leftarrow \mathcal{N}_f$;
16 $\mathcal{N}_f \leftarrow$ `empty;`
17 $\mathcal{N}_i^2 \leftarrow$ `empty;`
18 **for** *all* n_s *in* $\mathcal{N}'_f$ **do**
19 `expandRec(`n_s`);`
20 `procInflow(`$\mathcal{N}_i^2$, $step$`);`
21 `pruneGraph(`$\mathcal{G}$, $waveFront$, $thres$`);`
22 `updateEdges(`$\mathcal{G}$`);`
23 **return** $\mathcal{G}$, $\mathcal{N}_f$;

queue $\mathcal{N}_f$. During exploration, for each node in $\mathcal{N}_f$, its 8-neighborhoods is traversed to generate nodes associated with free grids in $Skeleton$, and neighbor nodes are designated as its child nodes (lines 15-19, Alg. 2). Nodes adjacent to unknown grids are added to $\mathcal{N}_f$, serving as starting points for the next expansion phase (line 5, Alg. 2). If a neighboring node $n_{s_{ni}}$ already exists, the pair $\{(n_{s_{ni}}, n_s)\}$ is added to the inflow junction candidates queue $\mathcal{N}_i^2$ (line 8, Alg. 2). Subsequently, **setNodeType(**n**)** is invoked to classify the node according to predefined criteria. All newly generated child nodes recursively follow this process to achieve depth-first graph expansion (lines 13-14, Alg. 2).

2) Simplification: We process inflow junction candidates first. Each pair in $\mathcal{N}_i^2$ is traced back towards their respective parent nodes within a specified step range $step$. If a common parent is found within $step$, the pair is identified as a fake inflow and no edge is established. Otherwise, one of the pair is designated as N_I, and the information of the relevant nodes is updated accordingly.

The uneven contours of obstacles can lead to artifacts or spurious features during skeleton extraction, potentially distorting the environment's topological structure, as shown in Fig. 2d. When two waves converge with a short propagation distance, it may indicate irregular obstacle shapes or sensor errors. Based on this assumption, if the wavefront value of newly generated N_T nodes is below the threshold $thres$, the entire branch is traced back and pruned until a N_B or N_C node is encountered.

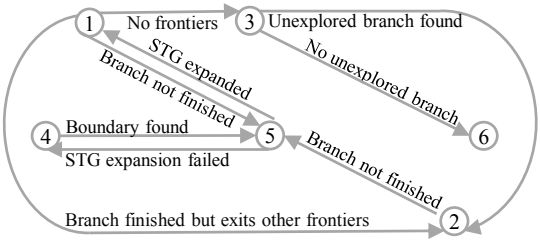


Fig. 3: Illustration of the exploration strategy FSM.

Edges are established between adjacent nodes in a parent-child hierarchy, while N_C nodes are omitted to simplify $\mathcal{G}$. Consequently, edges exist only between N_B, N_C, and N_T nodes, as illustrated in Fig. 2d. The sequence and length of the omitted N_C nodes are recorded as attributes of the edges, which are later used for path searching.

D. STG-based exploration strategy

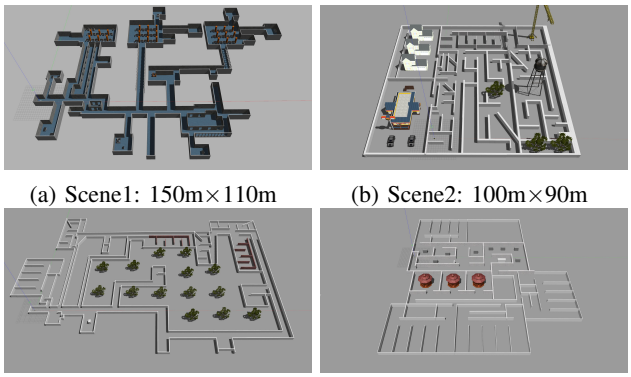
The STG inherently encodes information about frontiers, viewpoints, gains, and historical trajectories. Leveraging this, we propose a computationally efficient exploration strategy that eliminates the need for ray-casting and TSP-solving. This strategy is implemented through a FSM, as illustrated in Fig. 3, which comprises six distinct states.

State 1: Topology-Guided Exploration. This is the primary state of the exploration strategy. If $\mathcal{N}_f$ is not empty, classify all frontiers into branches based on their parent junction (N_I or N_B). If the current branch contains frontiers, transition to **State 5** with the nearest frontier to prioritize the completion of current branch, which effectively reducing BFMs. Otherwise, move to **State 2** to update the exploration branch. If $\mathcal{N}_f$ is empty, the exploration of the current region is complete, and the system transitions to **State 3** for backtracking. Moreover, STG positions targets centrally within paths, enhancing observation and navigation safety.

State 2: Exploration Branch Update. The exploration branch is updated once the current one is completed. Following the small-region-first approach outlined in [24], we greedily select the branch with the smallest wavefront value as the new exploration branch, as a smaller wavefront indicates a narrower region. Unselected branches are grouped and stored in the $missedBranches$ stack. The system then transitions to **State 5** with the nearest frontier in the updated exploration branch.

State 3: Backtracking. In this state, the system continuously pops entries from $missedBranches$ in chronological order to locate unexplored branches. Once a valid group is found, it transitions to **State 2** to update the exploration branch. If the stack is empty, it indicates that the environment has been fully explored, and the system transitions to the termination state, **State 6**.

State 4: Boundary-Guided Exploration. In overly open environments, the sensor's perception range limits the expansion of the STG, hindering the generation of effective frontier nodes. In this case, this state is activated to perform traditional boundary exploration. Following [18], the boundary between known and unknown regions is detected . A



(a) Scene1: 150m×110m (b) Scene2: 100m×90m

(c) Scene3: 210m×155m (d) Scene4: 130m×120m

Fig. 4: The four simulation environments.

2-opt heuristic algorithm is then applied to plan the path, transitioning to **State 5** with the first grid.

State 5: Planning and Moving. In this state, A* is employed for path planning based on the target s. If s is close to the agent and $state(skeleton, s) = unknown$, it indicates that the STG has not effectively expanded. Therefore, the system will transition to **State 4** to replan the target. Otherwise, the system will invoke the motion controller to proceed to the target and return to **State 1**. Furthermore, due to the structure of the STG, the backtracking phase allows for the rapid extraction of detailed paths from edges, significantly reducing the search space of A*.

State 6: Exploration Termination.

IV. EXPERIMENTAL RESULTS

A. Benchmark Comparison

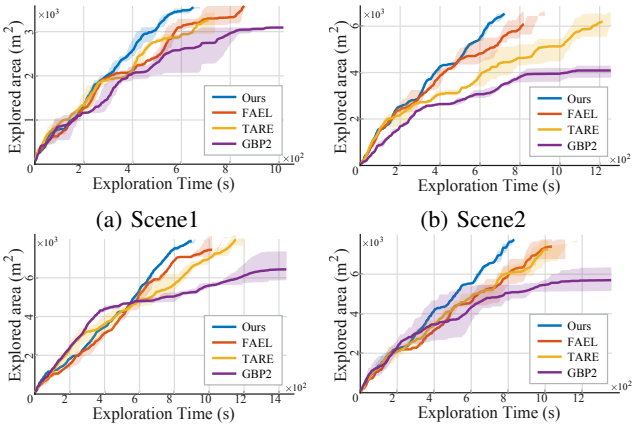
We conduct a series of experiments to evaluate the proposed method in various environments, as illustrated in Fig. 4. The comparative methods include sampling-based methods GBP2 [6] and TARE [11], as well as the frontier-based method FAEL [18], the latter two of which incorporates different forms of TSP solving. All methods are implemented using open-source code. To ensure a fair comparison of the exploration strategies, we standard the motion planner by using a modified version of [25] with $v_{max} = 2.0m/s$. We employ an open-source ground robot simulator [1], equipped with a Velodyne VLP-16 LiDAR with a FoV [360°, 30°], as the simulation platform. Each method is tested five times under identical configurations on a computer equipped with a 3.70 GHz Intel i9-10900K CPU running Ubuntu 20.04.

1) Exploration Performance: The quantitative results are shown in Table I, where bold represents the best. It is evident that our method demonstrates superior exploration performance in all four scenes, reducing exploration time by at least 19.8% and path length by at least 15.6%. To offer a more intuitive understanding of the performance, we also plot the curve of explored area over time, as shown in Fig. 5. In the initial phase, all methods demonstrate comparable performance, and our method occasionally underperforms, as observed in scene 1 and scene 3. However, as exploration continues, the advantages of our method become progressively more apparent.

[1] https://github.com/jackal/jackal

TABLE I: Comparison with SOTA methods.

Scene	Method	Exploration time(s)		Path length(m)		Run time(s)	Speed (m/s)
		Avg	Std	Avg	Std		
Scene1	GBP2 [6]	>1200	-	>954.4	-	0.337	0.80
	TARE [11]	733.8	54.9	1118.9	77.2	0.299	1.52
	FAEL [18]	786.4	92.0	1075.8	86.8	0.041	1.37
	Ours	**541.6**	18.4	**875.9**	33.5	**0.020**	**1.62**
Scene2	GBP2 [6]	>1500	-	>1411.1	-	0.795	0.95
	TARE [11]	1181.4	102.5	1716.6	21.7	0.375	1.45
	FAEL [18]	883.1	52.6	1266.8	38.6	0.082	1.52
	Ours	**681.4**	22.1	**1068.9**	3.8	**0.008**	**1.57**
Scene3	GBP2 [6]	>1700	-	>1915.8	-	0.377	1.12
	TARE [11]	1104.7	29.2	1889.4	44.8	0.312	**1.71**
	FAEL [18]	1020.7	59.7	1674.7	126.6	0.050	1.64
	Ours	**818.1**	21.2	**1372.4**	24.8	**0.033**	1.67
Scene4	GBP2 [6]	>1600	-	>1560	-	0.465	0.97
	TARE [11]	1018.6	82.3	1640.0	135.3	0.338	**1.61**
	FAEL [18]	1033.9	149.8	1420.3	128.4	0.119	1.37
	Ours	**779.8**	17.2	**1194.0**	12.9	**0.015**	1.53



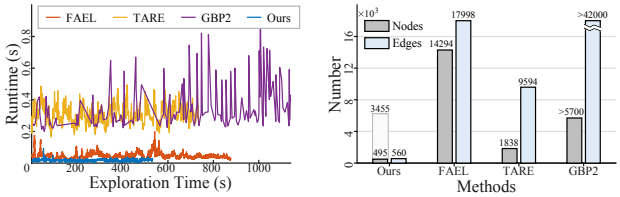
(a) Scene1 (b) Scene2

(c) Scene3 (d) Scene4

Fig. 5: The explored area over time. The shaded areas show the upper and lower bounds.

In the experiments of TARE and FAEL, the robot occasionally exhibited behaviors such as oscillation in place or BFMs at junctions and in narrow passages, which significantly impedes exploration efficiency. These behaviors stem from unreliable evaluation of viewpoint information gain under incomplete observations, which leads to instability in the 'optimal' trajectory solved by TSP. As the robot gathers new information during movement, the current optimal trajectory may conflict with the previously determined ones, especially in complex scenarios. This issue can also be observable in Fig. 5, where TARE and FAEL display more pronounced stepped profiles. This stepping pattern is due to leaving areas before they are fully explored, which necessitates frequent backtracking.

However, while our method, guided by the environmental topological structure, may not always achieve the maximum information gain at each stage, it ensures comprehensive exploration of the current branch in a single pass without inefficient BFMs. This method ultimately provides greater long-term benefits and results in improved exploration performance.



(a) Runtime of scene1. (b) Graph size of scene1.

Fig. 6: Illustration of computational performance.

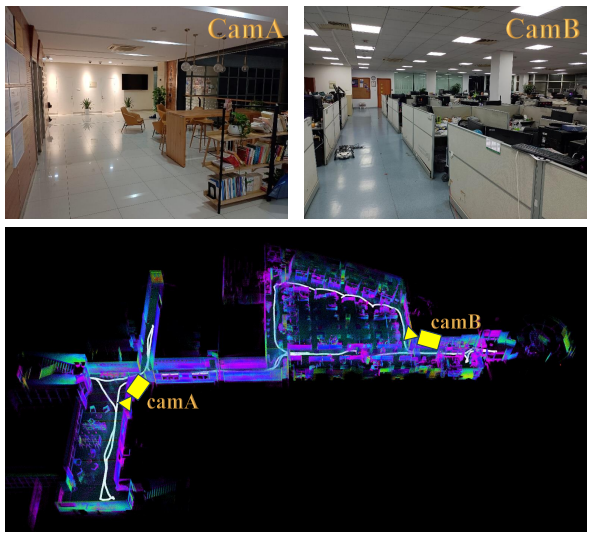
2) Computational Performance: Table I shows that our runtime significantly outperforms the compared methods. On average, STGPlanner demonstrates performance 4+ times faster than FAEL, 22+ times faster than TARE, and 38+ times faster than GBP2. The high runtime of these comparison methods primarily arises from three factors: solving the TSP, constructing RRTs, ray-casting-based gain evaluations, as noted in [8]. Additionally, in environments with narrow and branching structures, reducing the sampling step size to ensure comprehensive exploration further exacerbates TSP complexity and increases the frequency of ray-casting operations. In contrast, our method uses a lightweight STG to retain environmental information and employs a computationally efficient FSM to make exploration decisions based on the topological structure. Additionally, the tree-like growth structure of the STG significantly reduces the path search space during long-distance backtracking. These result in a significantly lower runtime throughout the exploration process.

We further record and plot the runtime during exploration in Scene1, as shown in Fig. 6a. GBP2 re-plans only after reaching last target, resulting in a low decision frequency. In later stages of exploration, it often oscillates between distant points with high computational overhead. In contrast, TARE and FAEL make decisions in real-time. The increase in candidate viewpoints in certain areas, such as junctions or expansive regions, significantly escalates the time required to solve the TSP. This contributes to the runtime spikes observed in Fig. 6a. Our method, on the other hand, maintains a consistently low runtime, with skeleton extraction requires approximately 12.3 ms, while STG updates and FSM decision-making take around 7.8 ms.

Moreover, We quantify the size of the graph generated by each method after completing the exploration in Scene1, as depicted in Fig. 6b. In our method, the gray bar represents N_B, N_I, and N_T nodes, while N_C nodes are shown separately as blank bar, since they are removed and do not contribute to edge construction. Compared to methods that generate roadmap through sampling, our proposed STG, which captures environmental topological characteristics, is significantly smaller in size. This reduction not only enhances exploration efficiency but also supports navigation tasks more effectively.

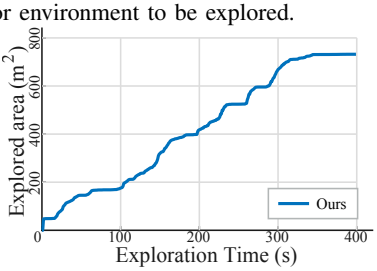
B. Real-World Experiment

We conduct real-world experiment using a quadrupedal robot platform equipped with a Jetson AGX Orin and a Livox



(a) The indoor environment to be explored.

(b) The platform.



(c) The explored area over time.

Fig. 7: Exploration experiment conducted in real-world.

Mid-360 LiDAR sensor in an indoor office environment. FAST-LIO2 [23] is utilized for providing localization information during the exploration process. The point cloud map established throughout the exploration and the exploration trajectory (white line) are depicted in Fig. 7a. The exploration of approximately $731m^2$ is completed in about 350s, with a total travel distance of 171m. This indicates that our proposed framework is able to perform environment exploration tasks on physical platforms with both safety and efficiency.

V. CONCLUSION

In this paper, we introduced STGPlanner, a novel autonomous exploration framework based on skeletal topological graph (STG). We proposed an incremental skeleton extraction method that allows the STG to expand effectively. Further, a FSM was designed to encapsulate the entire exploration strategy. By leveraging the topological structure of the environment, our method significantly reduces BFMs, enhancing exploration efficiency while minimizing computational costs. However, several limitations remain. In larger open environments, skeletal extraction may prove ineffective, necessitating a reliance on traditional frontier-based guidance methods. Furthermore, sensor noise and dynamic elements in real-world scenarios can adversely affect the skeletal structure. Future work will focus on adapting the system for dynamic environments and facilitating multi-robot cooperation.

REFERENCES

[1] B. Lindqvist, A. Patel, K. Löfgren, and G. Nikolakopoulos, "A tree-based next-best-trajectory method for 3-d uav exploration," *IEEE Transactions on Robotics*, vol. 40, pp. 3496–3513, 2024.

[2] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97. 'Towards New Computational Principles for Robotics and Automation'*, 1997, pp. 146–151.

[3] T. Cieslewski, E. Kaufmann, and D. Scaramuzza, "Rapid exploration with multi-rotors: A frontier selection method for high speed flight," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 2135–2142.

[4] D. Deng, R. Duan, J. Liu, K. Sheng, and K. Shimada, "Robotic exploration of unknown 2d environment using a frontier-based automatic-differentiable information gain measure," in *2020 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 2020, pp. 1497–1503.

[5] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon "next-best-view" planner for 3d exploration," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1462–1468.

[6] T. Dang, M. Tranzatto, S. Khattak, F. Mascarich, K. Alexis, and M. Hutter, "Graph-based subterranean exploration path planning using aerial and legged robots," *Journal of Field Robotics*, vol. 37, no. 8, pp. 1363–1388, 2020, wiley Online Library.

[7] H. Zhu, C. Cao, Y. Xia, S. Scherer, J. Zhang, and W. Wang, "Dsvp: Dual-stage viewpoint planner for rapid exploration by dynamic expansion," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 7623–7630.

[8] B. Tang, Y. Ren, F. Zhu, R. He, S. Liang, F. Kong, and F. Zhang, "Bubble explorer: Fast uav exploration in large-scale and cluttered 3d-environments using occlusion-free spheres," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 1118–1125.

[9] B. Kim, H. Seong, and D. H. Shim, "Topological exploration using segmented map with keyframe contribution in subterranean environments," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6199–6205.

[10] Y. Zhao, L. Yan, H. Xie, J. Dai, and P. Wei, "Autonomous exploration method for fast unknown environment mapping by using uav equipped with limited fov sensor," *IEEE Transactions on Industrial Electronics*, vol. 71, no. 5, pp. 4933–4943, 2024.

[11] C. Cao, H. Zhu, H. Choset, and J. Zhang, "Tare: A hierarchical framework for efficiently exploring complex 3d environments," in *Robotics: Science and Systems XVII*. Robotics: Science and Systems Foundation, 2021.

[12] X. Zhao, C. Yu, E. Xu, and Y. Liu, "Tdle: 2-d lidar exploration with hierarchical planning using regional division," in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, 2023, pp. 1–6.

[13] Y. Luo, Z. Zhuang, N. Pan, C. Feng, S. Shen, F. Gao, H. Cheng, and B. Zhou, "Star-searcher: A complete and efficient aerial system for autonomous target search in complex unknown environments," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4329–4336, 2024.

[14] J. A. Placed, J. Strader, H. Carrillo, N. Atanasov, V. Indelman, L. Carlone, and J. A. Castellanos, "A survey on active simultaneous localization and mapping: State of the art and new frontiers," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1686–1705, 2023.

[15] B. Zhou, H. Xu, and S. Shen, "Racer: Rapid collaborative exploration with a decentralized multi-uav system," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1816–1835, 2023.

[16] Z. Xu, C. Suzuki, X. Zhan, and K. Shimada, "Heuristic-based incremental probabilistic roadmap for efficient uav exploration in dynamic environments," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 832–11 838.

[17] B. Chen, Y. Cui, P. Zhong, W. Yang, Y. Liang, and J. Wang, "Stexplorer: A hierarchical autonomous exploration strategy with spatio-temporal awareness for aerial robots," *ACM Transactions on Internet Technology*, vol. 14, no. 6, Nov. 2023.

[18] J. Huang, B. Zhou, Z. Fan, Y. Zhu, Y. Jie, L. Li, and H. Cheng, "Fael: Fast autonomous exploration for large-scale environments with a mobile robot," *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1667–1674, 2023.

[19] Y. Hui, X. Zhang, H. Shen, H. Lu, and B. Tian, "Dppm: Decentralized exploration planning for multi-uav systems using lightweight information structure," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 613–625, 2024.

[20] Y. Tao, Y. Wu, B. Li, F. Cladera, A. Zhou, D. Thakur, and V. Kumar, "Seer: Safe efficient exploration for aerial robots using learning to predict information gain," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1235–1241.

[21] D. Duberg and P. Jensfelt, "Ufomap: An efficient probabilistic 3d mapping framework that embraces the unknown," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6411–6418, 2020.

[22] H. Choset and J. Burdick, "Sensor based planning. i. the generalized voronoi graph," in *Proceedings of 1995 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, 1995, pp. 1649–1655 vol.2.

[23] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Communications Of The ACM*, vol. 27, no. 3, p. 236–239, Mar. 1984.

[24] C. Wei, M. Xu, J. Wang, and Z. Chen, "Robot exploration based on small areas priority strategy," in *Proceedings of the 3rd International Conference on Service Robotics Technologies*, ser. ICSRT '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 7–14.

[25] J. Zhang, C. Hu, R. Chadha, and S. Singh, "Falco: Fast likelihood-based collision avoidance with extension to human-guided navigation," *Journal of Field Robotics*, vol. 37, Apr. 2020.